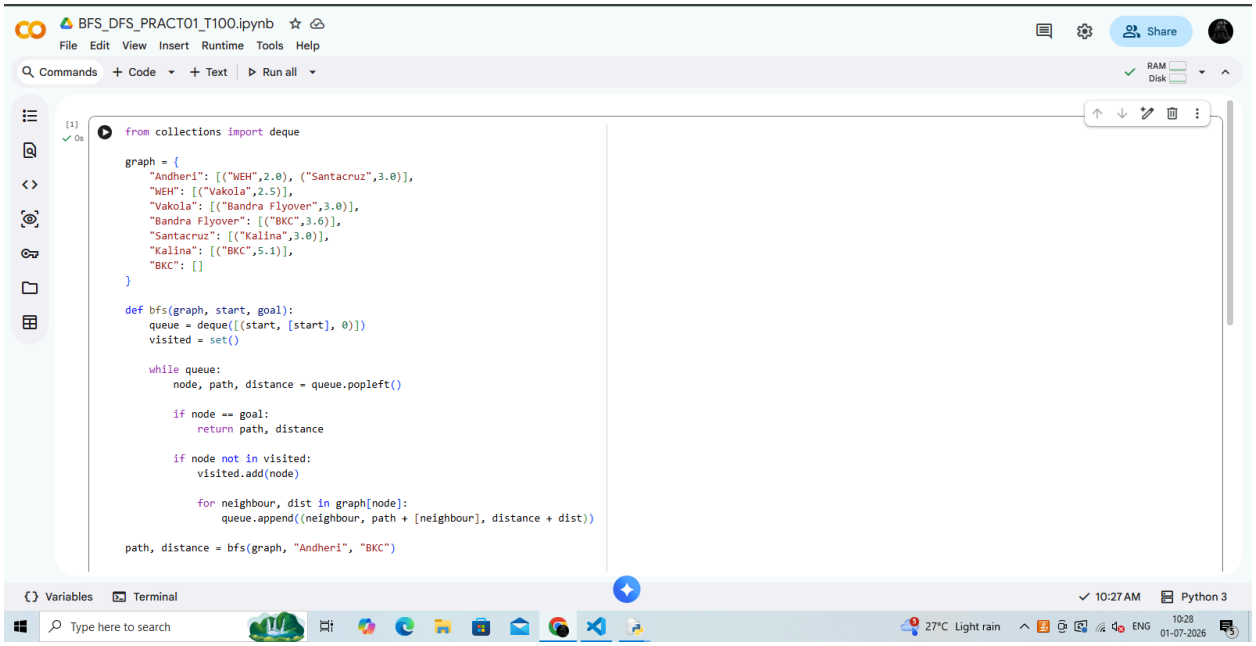
MVLU COLLEGE.

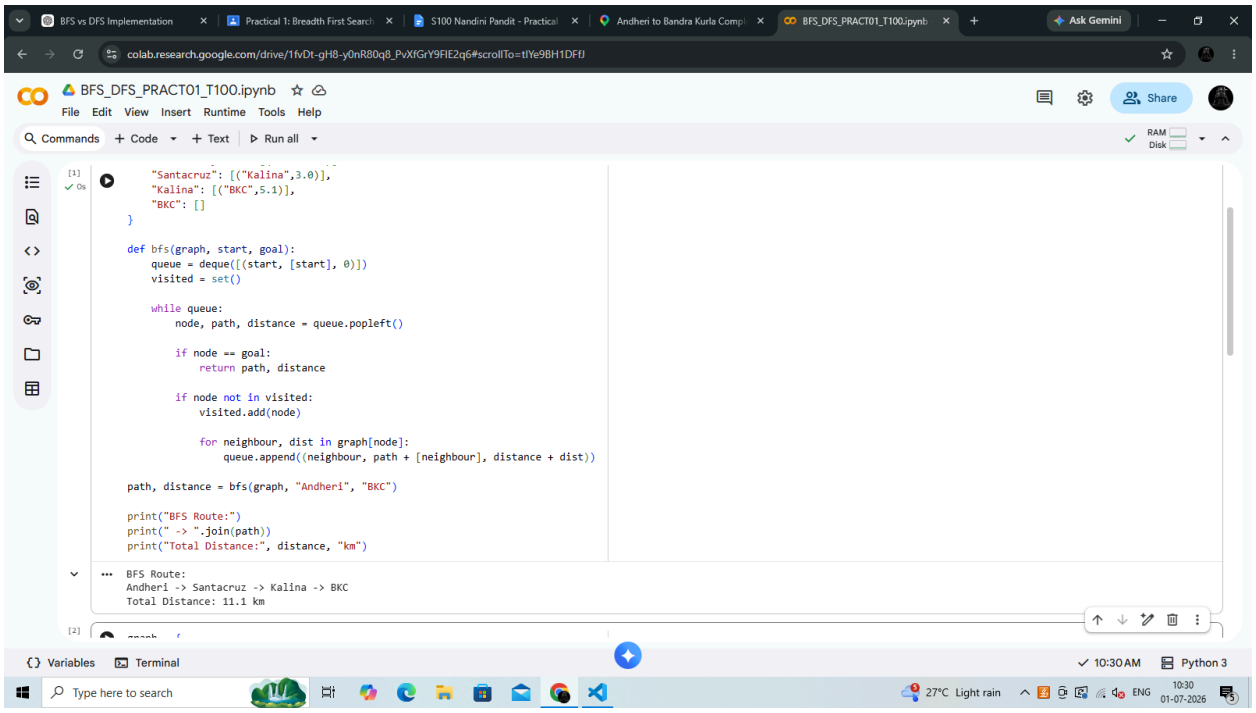
Aim :- A) Implement the Breadth First Search algorithm to solve a given problem.

```
from collections import deque

graph = {
    "Andheri": [("WEH",2.0), ("Santacruz",3.0)],
    "WEH": [("Vakola",2.5)],
    "Vakola": [("Bandra Flyover",3.0)],
    "Bandra Flyover": [("BKC",3.6)],
    "Santacruz": [("Kalina",3.0)],
    "Kalina": [("BKC",5.1)],
    "BKC": []
}

def bfs(graph, start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()

    while queue:
        node, path, distance = queue.popleft()

        if node == goal:
            return path, distance

        if node not in visited:
            visited.add(node)

            for neighbour, dist in graph[node]:
                queue.append((neighbour, path + [neighbour], distance +
dist))

path, distance = bfs(graph, "Andheri", "BKC")

print("BFS Route:")
print(" -> ".join(path))
print("Total Distance:", distance, "km")
```

NANDINI PANDIT T100
Artificial Intelligence.

MVLU COLLEGE.



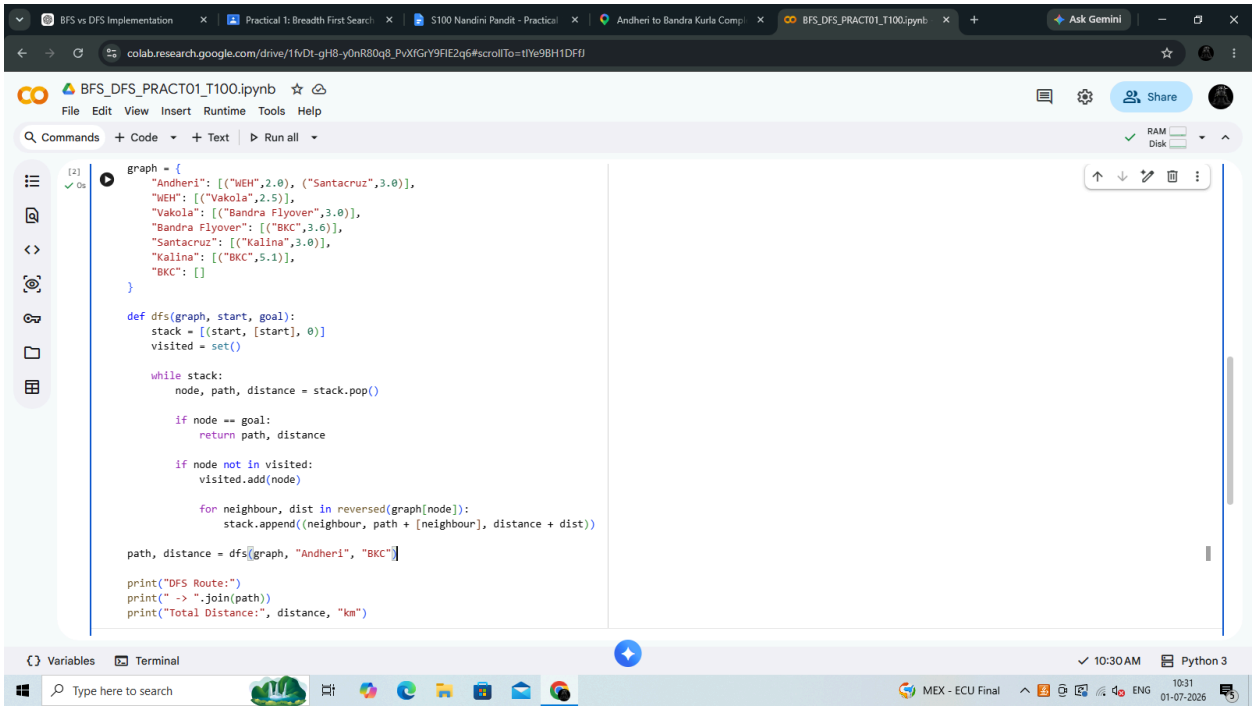


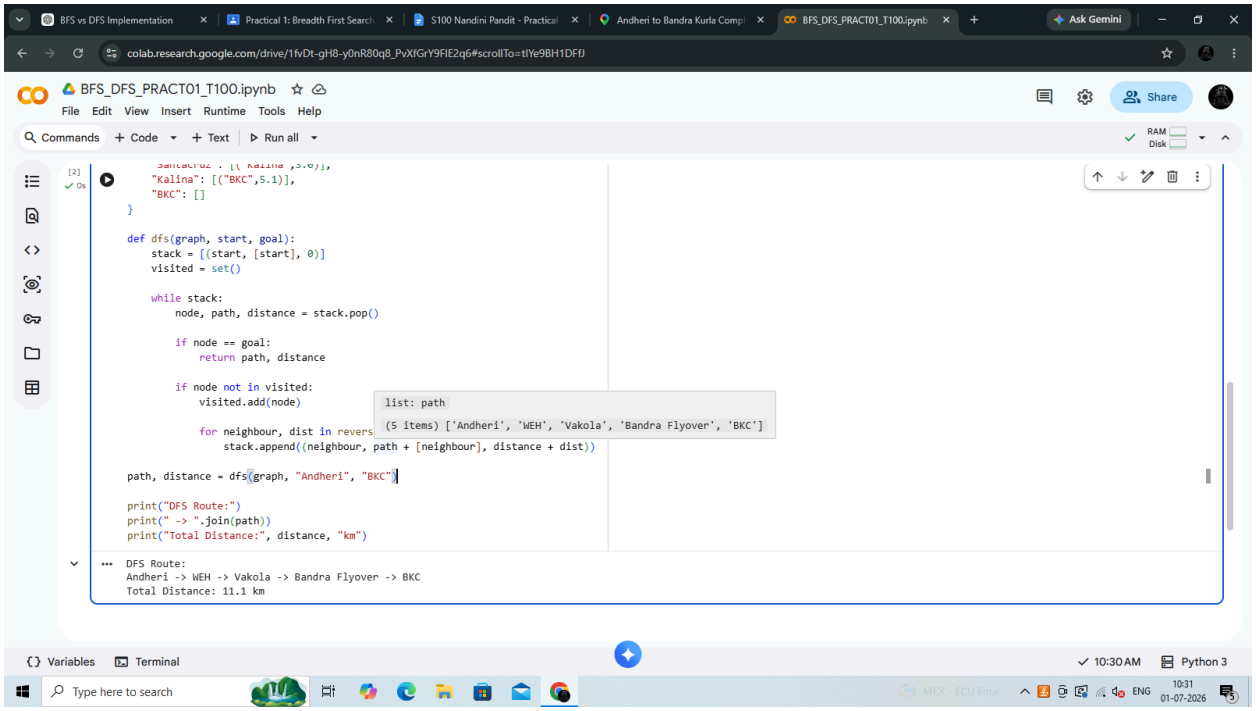
NANDINI PANDIT T100
Artificial Intelligence.

B) Implement the Iterative Depth First Search algorithm to solve the same problem.

```
graph = {
    "Andheri": [("WEH",2.0), ("Santacruz",3.0)],
    "WEH": [("Vakola",2.5)],
    "Vakola": [("Bandra Flyover",3.0)],
    "Bandra Flyover": [("BKC",3.6)],
    "Santacruz": [("Kalina",3.0)],
    "Kalina": [("BKC",5.1)],
    "BKC": []
}

def dfs(graph, start, goal):
    stack = [(start, [start], 0)]
    visited = set()

    while stack:
        node, path, distance = stack.pop()

        if node == goal:
            return path, distance

        if node not in visited:
            visited.add(node)

            for neighbour, dist in reversed(graph[node]):
                stack.append((neighbour, path + [neighbour], distance +
dist))

path, distance = dfs(graph, "Andheri", "BKC")

print("DFS Route:")
print(" -> ".join(path))
print("Total Distance:", distance, "km")
```

MVLU COLLEGE.



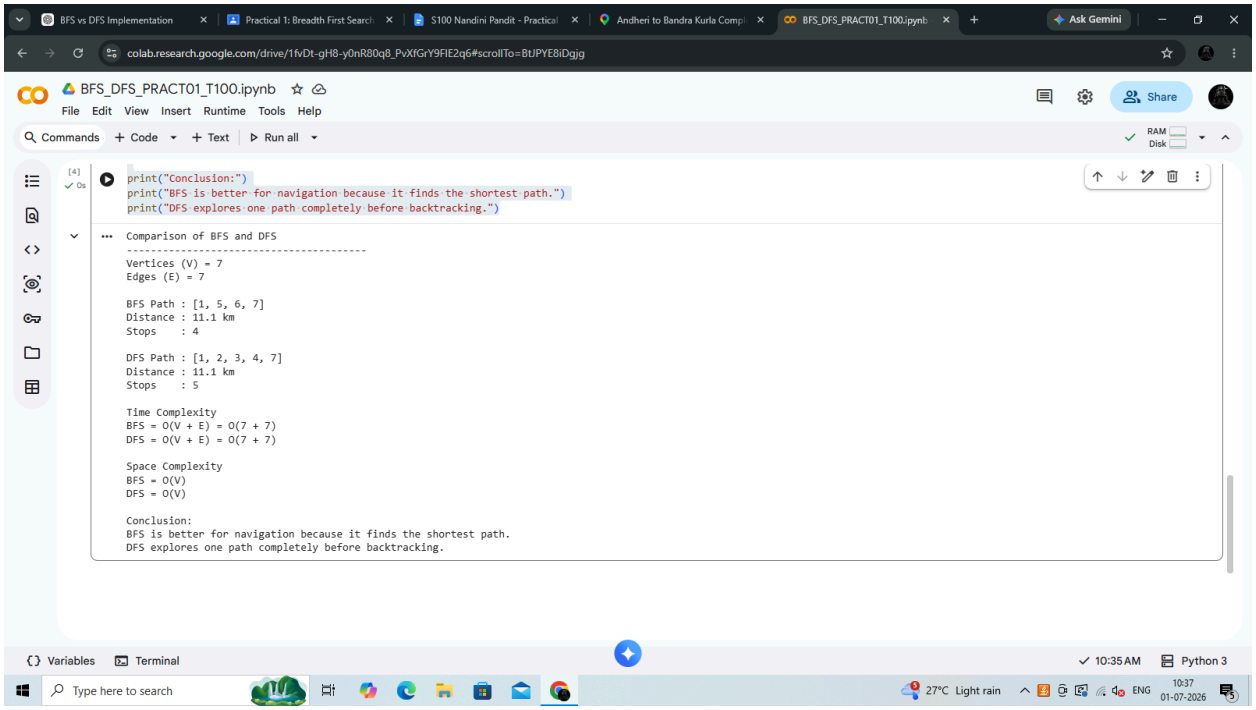


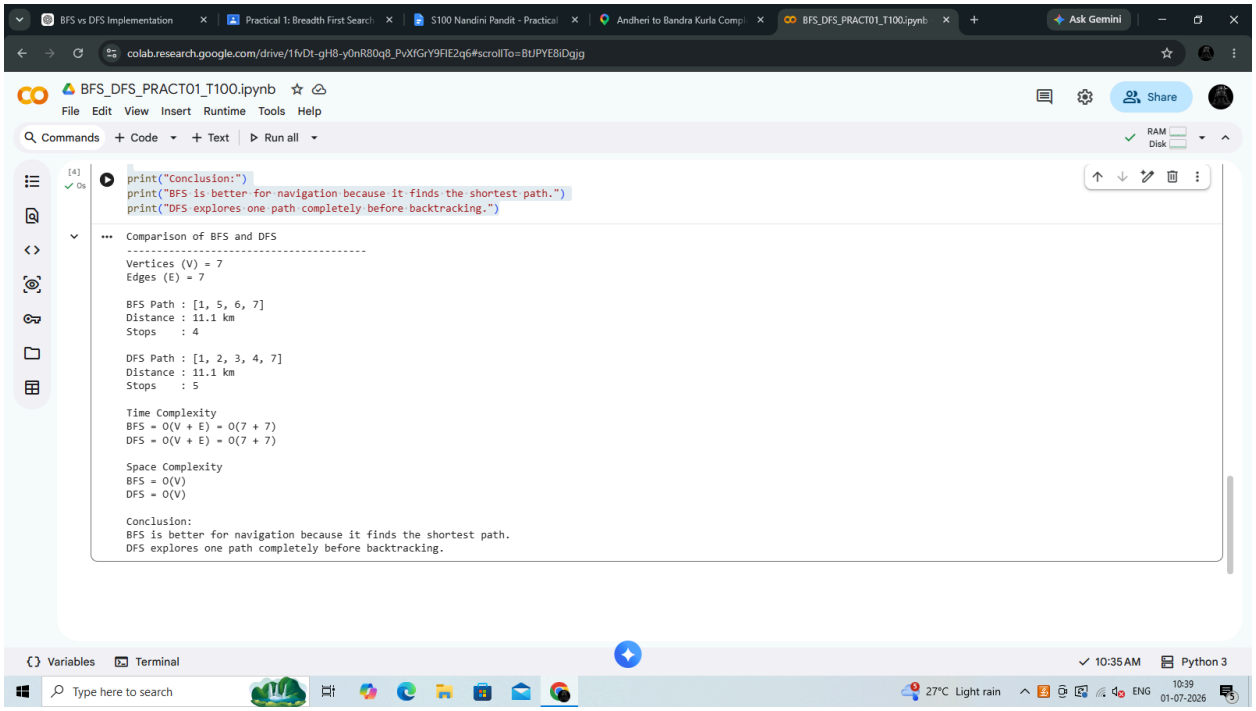
NANDINI PANDIT T100
Artificial Intelligence.

C) Compare the performance and efficiency of both algorithms.

```
print("Comparison of BFS and DFS")
print("-" * 40)

print("Vertices (V) = 7")
print("Edges (E) = 7\n")

print("BFS Path : [1, 5, 6, 7]")
print("Distance : 11.1 km")
print("Stops    : 4\n")

print("DFS Path : [1, 2, 3, 4, 7]")
print("Distance : 11.1 km")
print("Stops    : 5\n")

print("Time Complexity")
print("BFS = O(V + E) = O(7 + 7)")
print("DFS = O(V + E) = O(7 + 7)\n")

print("Space Complexity")
print("BFS = O(V)")
print("DFS = O(V)\n")

print("Conclusion:")
print("BFS is better for navigation because it finds the shortest path.")
print("DFS explores one path completely before backtracking.")
```

MVLU COLLEGE.





NANDINI PANDIT T100
Artificial Intelligence.